ESP32-S31 启动日志解析与硬件分析

https://gemini.google.com/app/b212c5e4511d3e7e

User prompt: 我可以使用 esp-idf 获复关于 ESP32-S31 的哪些在下面日志中显示了的信息 ESP-ROM:esp32s31-20251218 Build:Dec 18 2025 rst:0xc (SW_CPU_RESET),boot:0x58 (SPI_FAST_FLASH_BOOT) Core0 Saved PC:0x2f00409c Core1 Saved PC:0x2f0041e4 SPI mode:DIO, clock div:1 load:0x2f0740c0,len:0x168c load:0x2f06a2b0,len:0xc7c load:0x2f06cfb0,len:0x3370 entry 0x2f06a2ba I (32) boot: ESP-IDF v6.1 2nd stage bootloader I (32) boot: compile time Oct 7 2026 22:20:11 I (32) boot: Multicore bootloader I (34) boot: chip revision: v0.0 I (35) boot: efuse block revision: v0.1 I (38) boot.esp32s31: SPI Speed : 80MHz I (42) boot.esp32s31: SPI Mode : DIO I (46) boot.esp32s31: SPI Flash Size : 2MB I (50) boot: Enabling RNG early entropy source... I (54) boot: Partition Table: I (57) boot: ## Label Usage Type ST Offset Length I (63) boot: 0 nvs WiFi data 01 02 00009000 00006000 I (70) boot: 1 phy_init RF data 01 01 0000f000 00001000 I (76) boot: 2 factory factory app 00 00 00010000 00100000 I (84) boot: End of partition table I (86) esp_image: segment 0: paddr=00010020 vaddr=40018020 size=076c8h (30408) map I (99) esp_image: segment 1: paddr=000176f0 vaddr=2f000000 size=00928h (2344) load I (102) esp_image: segment 2: paddr=00018020 vaddr=40000020 size=11ee4h (73444) map I (119) esp_image: segment 3: paddr=00029f0c vaddr=2f000928 size=0a118h (41240) load I (126) esp_image: segment 4: paddr=0003402c vaddr=2f00aa80 size=024b8h (9400) load I (132) boot: Loaded app from partition at offset 0x10000 I (132) boot: Disabling RNG early entropy source... I (145) cpu_start: Multicore app I (154) cpu_start: GPIO 59 and 58 are used as console UART I/O pins I (154) cpu_start: Pro cpu start user code I (154) cpu_start: cpu freq: 320000000 Hz I (156) app_init: Application information: I (160) app_init: Project name: hello_world I (164) app_init: App version: 1 I (167) app_init: Compile time: Oct 7 2026 22:20:06 I (172) app_init: ELF file SHA256: 0d6e1323b... I (177) app_init: ESP-IDF: v6.1 I (180) efuse_init: Min chip rev: v0.0 I (184) efuse_init: Max chip rev: v0.99 I (188) efuse_init: Chip rev: v0.0 I (192) heap_init: Initializing. RAM available for dynamic allocation: I (198) heap_init: At 2F00E800 len 0006C7B0 (433 KiB): RAM I (204) heap_init: At 2F07AFB0 len 000041C0 (16 KiB): RAM I (209) heap_init: At 2E000000 len 00007FE8 (31 KiB): RTCRAM I (215) spi_flash: detected chip: generic I (218) spi_flash: flash io: dio W (221) spi_flash: Detected size(16384k) larger than the size in the binary image header(2048k). Using the size in the binary image header. I (233) sleep_gpio: Configure to isolate all GPIO pins in sleep state I (239) sleep_gpio: Enable automatic switching of GPIO sleep configuration I (246) main_task: Started on CPU0 I (246) main_task: Calling app_main() Hello world! This is esp32s31 chip with 2 CPU core(s), WiFi/BLE, 802.15.4 (Zigbee/Thread), silicon revision v0.0, 2MB external flash Minimum free heap size: 478208 bytes

Response: 通过你提供的启动日志，使用 **ESP-IDF（版本 v6.1）** 可以获取并确认关于 **ESP32-S31** 这款芯片的非常多关键硬件与软件信息。

以下是从日志中直接提取和分析出的各项信息：

1. 芯片基本硬件信息

- **芯片型号**：`esp32s31`（从 ROM 引导、`boot.esp32s31` 及最后的 `Hello world!` 输出中确认）。
- **CPU 架构与核心数**：多核心处理器，共有 **2 个 CPU 核心**（日志显示 `Core0` / `Core1` 寄存器状态，且 `cpu_start: Multicore app`，最终应用确认 `2 CPU core(s)`）。
- **CPU 运行频率**：主频为 **320 MHz**（`cpu freq: 320000000 Hz`）。
- **硅片版本（Silicon Revision）**：当前芯片版本为 **v0.0**（`chip revision: v0.0`，`efuse_init: Chip rev: v0.0`），支持的芯片修订版范围为 `v0.0` 到 `v0.99`。
- **eFuse 模块版本**：eFuse block revision 为 **v0.1**。
- **无线与通信能力**：支持 **WiFi / BLE** 以及 **802.15.4（支持 Zigbee / Thread）** 协议。

2. 外部 Flash 存储配置

- **Flash 型号/类型**：检测到通用闪存芯片 (`detected chip: generic`)，采用 **DIO 模式**（Dual I/O），时钟分频为 `1`。
- **SPI Flash 频率与模式**：SPI 速度为 **80 MHz**，模式为 **DIO**。
- **Flash 容量**：
  - 物理硬件实际检测到的 Flash 大小为 **16 MB**（`16384k`）。

- 但当前烧录的二进制镜像头部（Binary Image Header）声明的大小为 **2 MB**（2048k），系统当前优先按镜像头部的 2MB 进行限制和使用（见警告日志 `Using the size in the binary image header`）。

3. 内存（RAM）与堆内存分布

- **动态分配可用 RAM 总览**：
  - **主 RAM 区**：起始地址 `0x2F00E800`，长度 `0x0006C7B0`，约合 **433 KiB**。
  - **附加 RAM 区**：起始地址 `0x2F07AFB0`，长度 `0x000041C0`，约合 **16 KiB**。
  - **RTC RAM（低功耗内存区）**：起始地址 `0x2E000000`，长度 `0x00007FE8`，约合 **31 KiB**。
- **启动时的最小可用堆内存（Minimum free heap size）**：大约为 **478,208 字节**。

4. 引导与分区表配置（Partition Table）

日志中展示了完整的分区表结构：

1. **nvs**（WiFi 数据）：偏移量 `0x00009000`，长度 `0x00006000`。
2. **phy_init**（RF 射频校准数据）：偏移量 `0x0000f000`，长度 `0x00001000`。
3. **factory**（工厂应用程序主镜像）：从偏移量 `0x00010000` 开始，长度 `0x00100000`（约 1MB）。系统最终也是成功从该分区加载了应用程序（`Loaded app from partition at offset 0x10000`）。

5. 外设与低功耗配置

- **控制台串口（Console UART）**：系统配置使用 **GPIO 59 和 GPIO 58** 作为控制台的 UART 输入输出引脚（`GPIO 59 and 58 are used as console UART I/O pins`）。
- **睡眠/低功耗（Sleep GPIO）**：配置在睡眠状态下隔离所有 GPIO 引脚（`Configure to isolate all GPIO pins in sleep state`），并启用自动切换功能。
- **安全与随机数**：在启动早期启用了随机数生成器的熵源（`Enabling RNG early entropy source...`）。

6. 软件与编译信息

- **SDK 版本**：ESP-IDF **v6.1**（2nd stage bootloader 及应用层均确认）。
- **工程名称**：`hello_world`。
- **应用版本号**：`1`。
- **编译时间**：2026年10月7日 22:20:06（引导程序与主程序编译时间一致）。
- **ELF 文件哈希**：`0d6e1323b...`（用于固件防篡改及版本校验）。
- **复位原因（Reset Reason）**：`rst:0xc (SW_CPU_RESET)`，表明上一次复位是由 **软件 CPU 复位（Software CPU Reset）** 触发的。

User prompt: 请编写代码，调用 esp-idf API 尽量多的获取上述信息，复位原因（Reset Reason）与从什么加载程序除了显示代码，最好还要有对应的描述信息

Response: 下面为您编写了一段完整的 **ESP-IDF C 语言代码**。这段代码调用了 ESP-IDF 的各项系统 API，能够获取您在日志中看到的绝大部分核心信息（包括芯片型号、CPU 核心、版本、Flash 容量、运行分区、内存以及复位原因等），并且为**复位原因**和引导模式（Boot Mode）提供了详细的中文字面描述。

ESP-IDF 系统信息获取代码

您可以将以下代码放入您的 `main.c`（例如 `hello_world` 工程中）进行编译和运行：

```
#include <stdio.h>
#include "esp_system.h"
#include "esp_chip_info.h"
#include "esp_flash.h"
#include "esp_partition.h"
#include "esp_app_desc.h"
#include "esp_heap_caps.h"
#include "esp_idf_version.h"
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"

// 1. 复位原因 (Reset Reason) 的描述映射函数
const char* get_reset_reason_desc(esp_reset_reason_t reason) {
    switch (reason) {
        case ESP_RST_UNKNOWN:    return "未知原因 (Unknown)";
        case ESP_RST_POWERON:    return "上电复位：芯片刚刚通电 (Power-on event)";
        case ESP_RST_EXT:        return "外部引脚复位 (External pin reset)";
        case ESP_RST_SW:         return "软件复位：由代码或系统调用触发 (Software CPU reset)";
        case ESP_RST_PANIC:      return "异常崩溃复位：软件触发 Panic 或发生致命错误 (Software panic)";
        case ESP_RST_INT_WDT:    return "中断看门狗复位：CPU 中断响应超时 (Interrupt watchdog)";
        case ESP_RST_TASK_WDT:   return "任务看门狗复位：FreeRTOS 任务未及时喂狗 (Task watchdog)";
        case ESP_RST_WDT:        return "其他看门狗复位 (Other watchdog)";
        case ESP_RST_DEEPSLEEP:  return "从深度睡眠模式中唤醒 (Deep sleep wakeup)";
        case ESP_RST_BROWNOUT:   return "掉电复位：供电电压过低触发保护 (Brownout reset)";
        case ESP_RST_SDIO:       return "SDIO 接口复位 (SDIO reset)";
        default:                 return "未定义的复位类型";
    }
}

// 2. 引导模式 (Boot Mode / Boot Source) 的描述说明
// 注：日志中的 boot:0x58 (SPI_FAST_FLASH_BOOT) 由芯片 ROM 引导程序在硬件上通过 Strapping 引脚和配置决定
void print_boot_mode_description(void) {
    printf("\n--- 引导/加载程序来源说明 (Boot Mode Description) ---\n");
    printf("日志显示: boot:0x58 (SPI_FAST_FLASH_BOOT)\n");
    printf("详细含义解释:\n");
    printf("  * SPI Boot: 芯片从外部 SPI Flash 加载二级引导程序 (2nd stage bootloader)。\n");
    printf("  * FAST Flash Boot: 使用高速/快速 SPI 闪存启动模式（通常配合较高频的 SPI 时钟，如日志中的 80MHz）。\n");
    printf("  * 0x58 状态码: 由芯片内部 ROM 固件在读取 Strapping 管脚（如 GPIO 状态）后确定的引导源标志。\n");
}

void app_main(void) {
    printf("\n=======================================\n");
    printf("       ESP32-S31 系统信息诊断报告        \n");
    printf("=======================================\n");

    // --- 一、芯片硬件基本信息 ---
    esp_chip_info_t chip_info;
    esp_chip_info(&chip_info);

    printf("[硬件信息]\n");
    printf("  - 芯片型号 (Target)      : %s\n", CONFIG_IDF_TARGET);
    printf("  - CPU 核心数量           : %d 核\n", chip_info.cores);
    printf("  - 硅片硅版本 (Revision)  : v%d.%d\n", chip_info.revision / 100, chip_info.revision % 100);
    printf("  - 支持的无线/通信特性    : \n");
    printf("      * Wi-Fi              : %s\n", (chip_info.features & CHIP_FEATURE_WIFI_BGN) ? "支持" : "不支持");
    printf("      * BLE (蓝牙低功耗)   : %s\n", (chip_info.features & CHIP_FEATURE_BLE) ? "支持" : "不支持");
    printf("      * 802.15.4 (Zigbee)  : %s\n", (chip_info.features & CHIP_FEATURE_IEEE802154) ? "支持" : "不支持");

    // --- 二、复位原因信息 ---
    esp_reset_reason_t rst_reason = esp_reset_reason();
    printf("\n[复位状态]\n");
    printf("  - 复位原因代码 (Hex)     : 0x%x\n", rst_reason);
    printf("  - 复位原因详细说明       : %s\n", get_reset_reason_desc(rst_reason));

    // 调用引导模式解释
    print_boot_mode_description();

    // --- 三、应用与 SDK 软件信息 ---
    const esp_app_desc_t *app_desc = esp_app_get_description();
    printf("\n[软件与工程信息]\n");
    printf("  - 工程名称 (Project)     : %s\n", app_desc->project_name);
    printf("  - 应用版本 (Version)     : %s\n", app_desc->version);
    printf("  - 编译时间               : %s %s\n", app_desc->date, app_desc->time);
    printf("  - ESP-IDF 版本           : %s\n", esp_get_idf_version());

    // --- 四、Flash 存储容量 ---
    uint32_t flash_size = 0;
```

```
    if (esp_flash_get_size(NULL, &flash_size) == ESP_OK) {
        printf("\n[外部 Flash 存储]\n");
        printf("  - 检测到的 Flash 容量    : %lu MB (%lu 字节)\n", flash_size / (1024 * 1024), flash_size);
    }

    // --- 五、当前运行的分区 (Partition) ---
    const esp_partition_t *running = esp_ota_get_running_partition();
    if (running != NULL) {
        printf("\n[分区加载信息]\n");
        printf("  - 当前启动并运行的分区名 : %s\n", running->label);
        printf("  - 分区类型 (Type/Subtype): 0x%02x / 0x%02x\n", running->type, running->subtype);
        printf("  - 分区在 Flash 中的偏移  : 0x%08lx\n", running->address);
        printf("  - 分区大小               : 0x%08lx 字节\n", running->size);
    }

    // --- 六、堆内存 (Heap) 统计 ---
    printf("\n[内存堆 (Heap) 状态]\n");
    printf("  - 当前可用空闲堆内存     : %u 字节\n", (unsigned int)esp_get_free_heap_size());
    printf("  - 启动以来的最小空闲内存 : %u 字节\n", (unsigned int)esp_get_minimum_free_heap_size());

    printf("\n=======================================\n");
}
```

关键信息的 API 对应关系与描述解析

1. **复位原因（Reset Reason）**
   - **API**：`esp_reset_reason()`
   - **代码中的对应**：返回 `esp_reset_reason_t` 枚举。日志中的 `rst:0xc (SW_CPU_RESET)` 对应的就是 `ESP_RST_SW`（软件 CPU 复位），即上面函数中解释的 **“由代码或系统调用触发的软件复位”**。
2. **引导加载来源（Boot Mode / Boot Source）**
   - **日志表现**：`boot:0x58 (SPI_FAST_FLASH_BOOT)`
   - **描述**：这是由芯片上电时硬件引脚（Strapping Pins）的电平状态决定的。`0x58` 代表芯片成功识别到外接了 SPI 闪存，并采用**高速 DIO/FAST 模式**去引导并加载二级引导程序。通常由内部 ROM 固件直接处理，应用层可以通过上述的说明函数理解其硬件含义。
3. **当前运行的分区（Running Partition）**
   - **API**：`esp_ota_get_running_partition()`
   - **描述**：对应日志中的 `Loaded app from partition at offset 0x10000`（即 `factory` 分区），它能够自动返回当前系统正在执行的主程序是从哪个 Flash 偏移地址加载进来的。
